

任务三项目测试报告

任务三是使用 mini-c 语言的正则表达式和文法对任务一、任务二的项目分别进行测试，具体如下：

1. 任务一测试

1.1 数据准备

构建 mini-c 语言的正则表达式如下：

```
digit=[0-9]
letter=[a-zA-Z]
_key100S=else|if|int|float|double|return|void|do|while
_special200S=\+|-
|\*|/|%|<|<=|>|=|!=|=|;|,|\(|\)|\\|\"|'|{|}
_specialName200S=PLUS|MINUS|MULTIPLY|DIVIDE|MOD|LT|LTEQ|RTEQ|RT|EQ|
NE|ASSIGN|SEMI|DOU|LLM|RLM|LMM|RMM|LBM|RBM
_ID400=(|letter)(|letter|digit)*
_NUMBER500=digit+(.digit)?
_annotation600=//~\n
```

构建的_simple.tny 代码如下：

```
// xxxxx 注释示例

int max ( int x, int y ) {
    if ( x > y ) {
        return x ;
    } else {
```

```
        return y ;

    }

}

int x ;
int fact ;
double y ;
int arr [ 100 ] ;
void test ( int a , int b )
{
    a = 1 ;
    b = 2 ;
    return ;
}
void main ( void )
{
    x = 2 ;
    y = 3 ;
    if ( x > 1 )
        do
            fact = fact * x ;
        while ( fact >= 1 ) ;
    else
        x = 1 ;
    return ;
}
```

1.2 测试过程

启动项目后，输入上面准备的 mini-c 语言的正则表达式，点击“开始分析”按钮进行分析，之后再依次点击“NFA 生成”、“DFA 生成”、“DFA 最小化”、“词法分析程序生成”对各项结果进行查看：

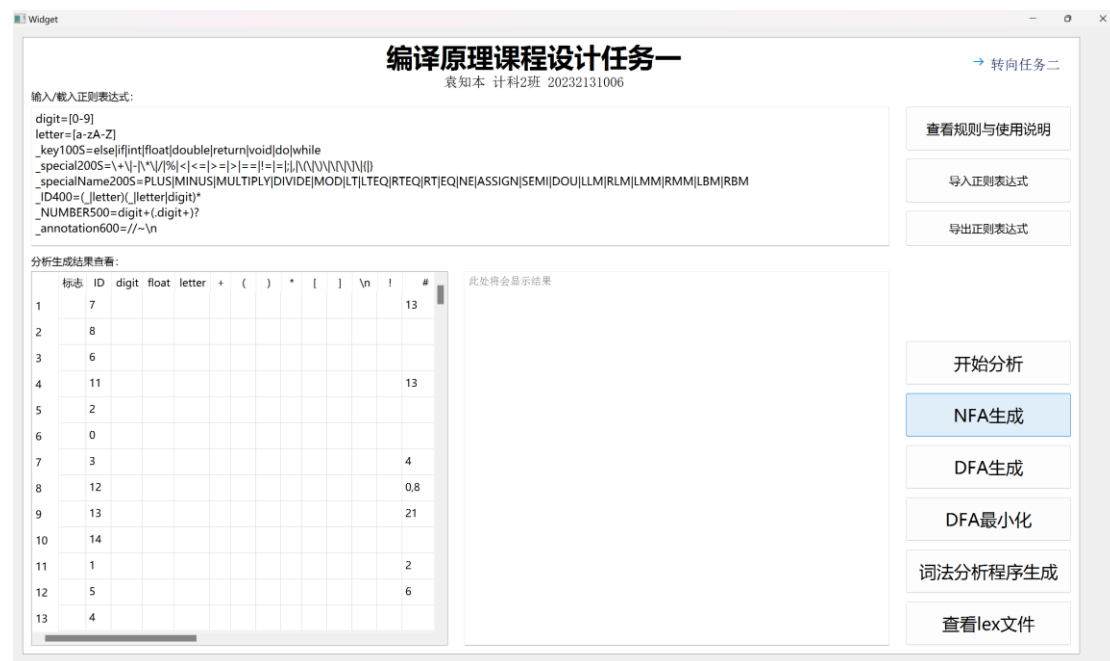


图 1 生成 NFA

由于显示区域大小问题，构建的表格形式展示的 NFA 图无法显示完全，但是可以通过滑动条实现完整的查看。

图 3 DFA 最小化

由于显示区域大小问题，构建的表格形式展示的最小化 DFA 图无法显示完全，但是可以通过滑动条实现完整的查看。

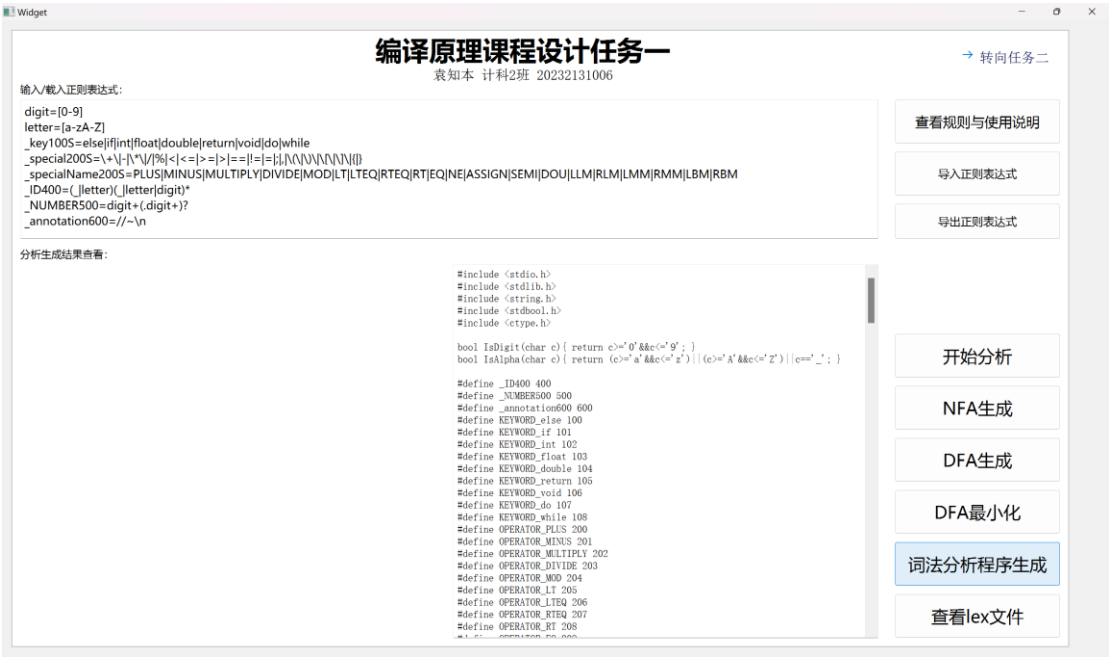


图 4 词法分析程序生成

由于显示区域大小问题，生成的语法分析程序无法显示完全，但是可以通过滑动条实现完整的查看。

通过指令【`gcc _lexer.c -o _lexer.exe`】编译该保存下来的`_lexer.c`文件后，生成了可执行程序`_lexer.exe`，此时将测试用的`sample.tny`放到同一文件夹下，

运行可执行程序，即可得对应的 output.lex 编码程序文件，再通过任务一项目系统的“查看 lex 文件”按钮实现查看 output.lex 文件内容：



图 5 lex 文件查看

1.3 测试结果

经过测试，mini-c 正则表达式输入后测试所有的功能均通过，系统能实现对 mini-c 正则表达式的分析并生成 NFA、DFA 图（以表格形式展示），进行 DFA 图最小化并展示（表格形式），生成对应的语法分析程序，并能对 lex 文件进行查看，全部测试都能通过。

2. 任务二测试

2.1 数据准备

由于需要对任务二系统的文法输入格式进行适配，故我对 mini-c 语言的文法格式做了少量的修改，但不影响 mini-c 文法的功能，具体最终使用的 mini-c 文法如下：

```
program|definition-list|definition|global-variable-  
definition|function-definition|type-indicator|parameters|compound-  
stmt|parameter-list|parameter|local-definition-list|local-  
definition|statement-list|statement|matched-stmt|unmatched-  
stmt|expression-stmt|condition-stmt|return-  
stmt|expression|assignment-expression|simple-  
expression|variable|additive-  
expression|relop|term|addop|mulop|factor|call|arguments|argument-  
list|while-stmt|do-while-stmt|for-stmt|for-init|for-condition|for-  
update  
  
ID|SEMI|LMM|RMM|LBM|RBM|int|float|double|void|LLM|RLM|if|else|do|wh  
ile|for|return|LTEQ|LT|RT|RTEQ|EQ|NE|PLUS|MINUS|MULTIPLY|DIVIDE|MOD  
|NUMBER|FLOAT_NUMBER|DOU|ASSIGN  
  
program -> definition-list  
  
definition-list -> definition-list definition  
  
definition-list -> definition  
  
definition -> global-variable-definition  
  
definition -> function-definition  
  
global-variable-definition -> type-indicator ID ASSIGN NUMBER SEMI  
  
global-variable-definition -> type-indicator ID ASSIGN FLOAT_NUMBER  
SEMI  
  
global-variable-definition -> type-indicator ID SEMI  
  
global-variable-definition -> type-indicator ID LMM NUMBER RMM SEMI  
  
global-variable-definition -> type-indicator ID ASSIGN simple-  
expression SEMI  
  
type-indicator -> int
```

type-indicator -> float
type-indicator -> double
type-indicator -> void
function-definition -> type-indicator ID LLM parameters RLM
compound-stmt
parameters -> parameter-list
parameters -> void
parameters -> #
parameter-list -> parameter-list DOU parameter
parameter-list -> parameter
parameter -> type-indicator ID
parameter -> type-indicator ID LMM RMM
compound-stmt -> LBM local-definition-list statement-list RBM
local-definition-list -> local-definition-list local-definition
local-definition-list -> #
local-definition -> type-indicator ID ASSIGN NUMBER SEMI
local-definition -> type-indicator ID ASSIGN FLOAT_NUMBER SEMI
local-definition -> type-indicator ID SEMI
local-definition -> type-indicator ID LMM NUMBER RMM SEMI
local-definition -> type-indicator ID ASSIGN simple-expression SEMI
statement-list -> statement-list statement
statement-list -> #
statement -> matched-stmt
statement -> unmatched-stmt
statement -> while-stmt
statement -> for-stmt
statement -> do-while-stmt


```
matched-stmt -> expression-stmt
matched-stmt -> compound-stmt
matched-stmt -> return-stmt
matched-stmt -> while-stmt
matched-stmt -> for-stmt
matched-stmt -> do-while-stmt
matched-stmt -> if LLM expression RLM matched-stmt else matched-
stmt
unmatched-stmt -> if LLM expression RLM statement
unmatched-stmt -> if LLM expression RLM matched-stmt else
unmatched-stmt
expression-stmt -> expression SEMI
expression-stmt -> SEMI
return-stmt -> return SEMI
return-stmt -> return expression SEMI
while-stmt -> while LLM expression RLM statement
do-while-stmt -> do statement while LLM expression RLM SEMI
for-stmt -> for LLM for-init SEMI for-condition SEMI for-update RLM
statement
for-init -> expression
for-init -> #
for-condition -> expression
for-condition -> #
for-update -> expression
for-update -> #
expression -> assignment-expression
expression -> simple-expression
```

```
assignment-expression -> variable ASSIGN expression
variable -> ID
variable -> ID LMM expression RMM
simple-expression -> additive-expression relop additive-expression
simple-expression -> additive-expression
relop -> LTEQ
relop -> LT
relop -> RT
relop -> RTEQ
relop -> EQ
relop -> NE
additive-expression -> additive-expression addop term
additive-expression -> term
addop -> PLUS
addop -> MINUS
term -> term mulop factor
term -> factor
mulop -> MULTIPLY
mulop -> DIVIDE
mulop -> MOD
factor -> LLM expression RLM
factor -> variable
factor -> call
factor -> NUMBER
factor -> FLOAT_NUMBER
call -> ID LLM arguments RLM
```

```
arguments -> argument-list
arguments -> #
argument-list -> argument-list DOU expression
argument-list -> expression
```

用于测试的 mini-c 代码对应的 lex 文件内容如下所示：

```
102 400 "max" 214 102 400 "x" 213 102 400 "y" 215 218 101 214
400 "x" 208 400 "y" 215 218 105 400 "x" 212 219 100 218 105 400
"y" 212 219 219 102 400 "x" 212 102 400 "fact" 212 104 400 "y"
212 102 400 "arr" 216 500 "100" 217 212 106 400 "test" 214 102
400 "a" 213 102 400 "b" 215 218 400 "a" 211 500 "1" 212 400 "b"
211 500 "2" 212 105 212 219 106 400 "main" 214 106 215 218 400
"x" 211 500 "2" 212 400 "y" 211 500 "3" 212 101 214 400 "x" 208
500 "1" 215 107 400 "fact" 211 400 "fact" 202 400 "x" 212 108
214 400 "fact" 207 500 "1" 215 212 100 400 "x" 211 500 "1" 212
105 212 219 -1
```

2.2 测试过程

编译原理课程设计任务二
袁知本 计科2班 20232131006

文法输入编辑
查看输入规则 打开 保存

文法分析
查看分析说明 LR(0) DFA生成 SLR(1) 文法分析
LR(1) DFA生成 LR(1) 分析表生成 源代码分析

语法规则生成 代码生成 可视化语法树

使用说明 代码生成 语法树展示

本模块用于可视化语法树构建和中间代码生成，具体步骤如下：
1. 先在本系统的左侧的文法模块中输入文法
2. 再在本系统的右侧的句子模块中输入该文法对应的测试源代码
3. 完成以上步骤后，点击代码生成，即可进行中间代码生成；点击可视化语法树，可以实现语法树构建，可分别通过点击上面“代码生成”和“语法树展示”按钮查看。

分析结果表

状态	状态内文法	ASSIGN	DIVIDE	DOU	EQ	FLC
19	factor->LLM.expressionRMLM ...					29
20	global-variable-definition->type...					
21	simple-expression->additive...				40	
22	factor->call.					
23	term->factor.					
24	global-variable-definition->type...					
25	additive-expression->term. term...			51		
26	factor->variable.					
27	global-variable-definition->type...					
28	call->IDLMM.argumentsRMLM ...					29
29	variable->IDLMM.expressionRMM ...					29
30	factor->FLOAT.NUMBER					

图 6 FIRST、FOLLOW 集构建、LR(0)DFA 构建

如图 6 所示，在“文法载入编辑”框内输入/导入此前得到的 mini-c 文法规则文本，点击下方“First 集合求解”即可查看到基于该文法构建的 First 集合（受限显示框大小，无法将 First 集完整显示，但可以通过滚动条拉动实现完整的查看），点击下方的“Follow 集合求解”则可查看到基于该文法构建的 Follow 集合（受限显示框大小，无法将 Follow 集完整显示，但可以通过滚动条拉动实现完整的查看）

再点击系统界面正上方的“LR(0)DFA 生成”按钮，即可在下方的“分析结果表”中查看到基于当前文法构建的表格形式展示的 LR(0)DFA（同样受制于显示框无法完整展示，但可以通过滚动条实现完整查看）。同时，在“文法分析”栏处看到文法的输出进行对比参照。

分析结果表

	状态	状态内文法	ASSIGN	DIVIDE	DOU	EQ	FLC
19	18	factor->LLM.expressionRLM , SEMI ...					46
20	19	global-variable-definition->type-...					
21	20	simple-expression->additive-...				61	
22	21	factor->call. , SEMI factor->call. , E...					
23	22	term->factor. , SEMI term->factor....					
24	23	global-variable-definition->type-...					
25	24	additive-expression->term. , SEMI ...		92			
26	25	factor->variable. , SEMI factor-...					
27	26	global-variable-definition->type-...					
28	27	call->IDLLM.argumentsRLM , SEMI ...					29
29	28	variable->IDLMM.expressionRMM , ...					75
30	29	factor->FLOAT.NUMBER , RLM					

图 7 LR(1)DFA 构建

点击图 6 中的“LR(1)DFA 生成”，即可对当前输入的 mini-c 文法进行表格形式的 LR(1)DFA 生成，如图 7 所示（同样受制于显示框无法完整展示，但可以通过滚动条实现完整查看）。

分析结果表

	状态	\$	ASSIGN	DIVIDE	DOU	E	LLM	L
1	0							
2	1	r2						
3	2	acc						
4	3						r12	
5	4						r11	
6	5	r4						
7	6	r3						
8	7						r10	
9	8						s11	
10	9						r13	
11	10	r1						

提示 LR(1)分析表生成完成! OK

图 8 LR(1)分析表构建

点击图 6 中的“LR(1)分析表生成”，即可对当前输入的 mini-c 文法进行对应的 LR(1)分析表构建，具体构建结果如图 8 所示（同样受制于显示框无法完整展示，但可以通过滚动条实现完整查看）



图 9 载入 LEX，进行源程序语法分析

如图 9 所示，在项目系统界面左侧“lex 载入编辑”框内输入/载入 mini-c 测试源代码通过任务一生成的词法分析程序分析得到的 output.lex 文件内容，之后点击系统界面正上方的“源代码分析”按钮，即可在下方的“分析结果表”中查看到 mini-c 测试源代码的语法分析逐步过程。

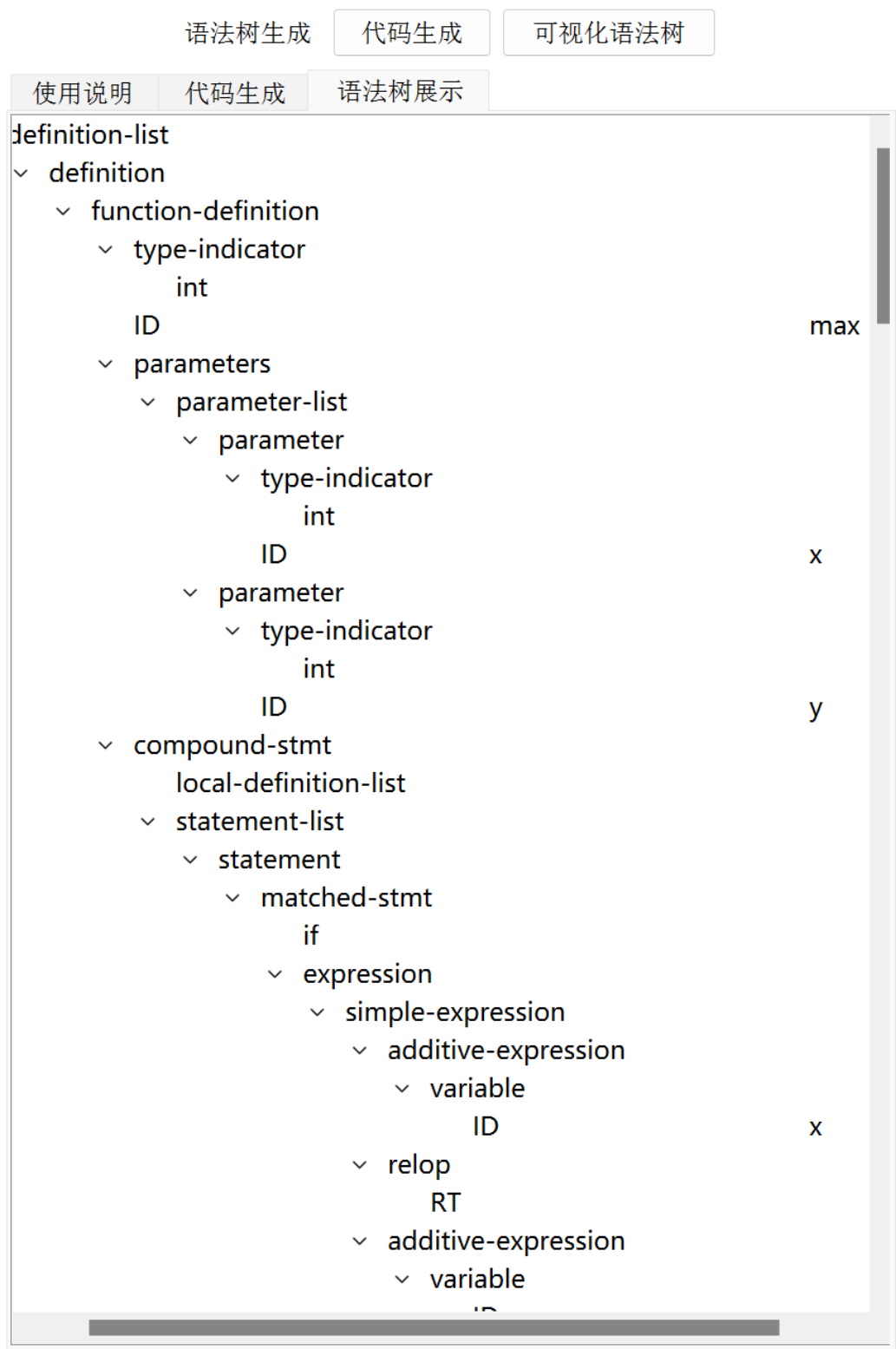


图 10 语法树生成

在完成 mini-c 测试源代码的语法分析后，点击系统界面右上角的“可视化语法树”按钮，即可生成语法树并通过下方的“语法树展示”区进行展示，如图 10 所示。

据上述测试流程可知，基于数据准备中所提到的 mini-c 文法和 mini-c 测试程序源代码，把此二者输入后，依次点击按钮控件进行 FIRST 集合求解、FOLLOW 集合求解、LR(0)DFA 构建、SLR(1)文法判断、LR(1)DFA 生成、LR(1)分析表构建、源代码分析、语法树可视化以及中间代码生成。

经过测试，FIRST 集、FOLLOW 集、LR(0)DFA、LR(1)DFA、LR(1)分析表、源程序语法分析表、可视化语法树均能正确生成，由于显示区域的大小问题，这几项功能都无法在初始情况下将结果完整展示出来，但可以通过上下、左右的滚动条实现完整查看。

对于 SLR(1)语法判断分析，出现了规约-规约冲突、移进-规约冲突，对于移进-规约冲突，采取只做移进不做规约处理；对于规约-规约冲突，采用选择第一个匹配的产生式处理。

2.3 测试结果

经过测试，mini-c 文法和 mini-c 测试源码输入后测试所有的功能均通过，系统能实现对 mini-c 文法和测试源码进行分析并构建 FIRST 集、FOLLOW 集、LR(0)DFA（表格形式展示）、LR(1)DFA（表格形式展示）、SLR(1)文法判断分析、

LR(1)分析表、句子分析表、可视化语法树、中间代码，全部测试均可通过。